

SENA · PIEDECUESTA — CURSO DE INTELIGENCIA ARTIFICIAL

Clase alternativa (avanzada) App React con un agente LLM y base de datos vectorial (RAG)

Guía de trabajo del aprendiz · React + Express + Cohere + Supabase +
Gemini

Duración: 4 horas (8:00 am – 12:00 m)

Clase alternativa (avanzada)

Pablo Andrés Carvajal — Desarrollador Full Stack

pablocarvajal.dev

Nombre del aprendiz: _____

🎯 ¿Qué sabré hacer al terminar esta guía?

- Explicar la **arquitectura** de una app RAG: React → backend → embeddings + base vectorial + LLM.
- Justificar **por qué se necesita un backend** para proteger las claves de API.
- Crear un proyecto **React con Vite** y un **backend Express**.
- Configurar **Supabase con pgvector**: tabla de documentos y función de búsqueda por similitud.
- Implementar la **ingesta** (chunking + embeddings con Cohere) y la **consulta** (retrieval + Gemini).
- Conectar el **chat de React** con el backend y desplegar la app.

LA GRAN INTEGRACIÓN

Esta clase reúne todo el curso en un proyecto real: los **embeddings** y el **RAG** de la Clase 2, los **agentes** de la Clase 4 y las **APIs gratuitas** de la Clase 0, ahora dentro de una **app web con React** que puedes publicar en internet.

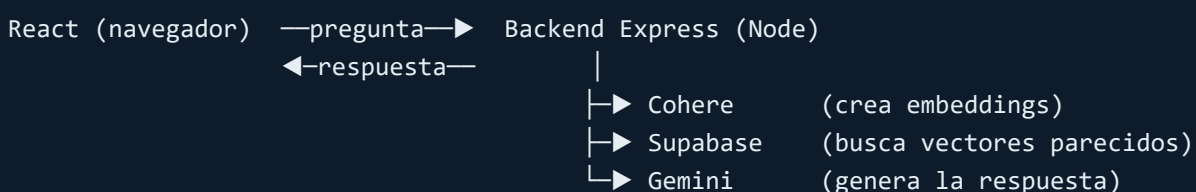
IMPORTANTE ANTES DE EMPEZAR

Los **nombres de modelos y SDK** (por ejemplo `gemini-1.5-flash`, `embed-multilingual-v3.0`) y los **planes gratuitos** cambian con el tiempo. Si algo no funciona, revisa la **documentación oficial** de Google AI Studio, Cohere y Supabase para ver la versión vigente.

1. Qué vamos a construir y por qué así

Vamos a construir un **chat en React** que responde preguntas usando **nuestros propios documentos** (apuntes, reglamento, manual de un producto...). Esta técnica se llama **RAG** (Retrieval-Augmented Generation): en lugar de confiar en lo que el modelo "recuerda", le **entregamos la información** justo antes de responder, así contesta con datos reales y **alucina mucho menos**.

La arquitectura



El stack tecnológico

Pieza	Herramienta	Para qué sirve
Frontend	React + Vite	La interfaz de chat en el navegador
Backend	Node.js + Express	Proteger las claves y orquestar el RAG
Embeddings	Cohere <code>embed-multilingual-v3.0</code> (1024 dimensiones)	Convertir texto en vectores
Base vectorial	Supabase + extensión pgvector	Guardar y buscar los vectores
LLM	Google Gemini <code>gemini-1.5-flash</code>	Generar la respuesta final

ALTERNATIVAS DE BASE VECTORIAL

Si no quieres usar Supabase, puedes usar **Pinecone** o **Upstash Vector** (en la nube) o **Chroma** (local, en tu propio PC). La idea de "guardar embeddings y buscar los más parecidos" es la misma.

2. Por qué se necesita un backend (seguridad de claves)

El código de React **se descarga completo al navegador del usuario**. Cualquiera puede abrir las herramientas de desarrollo (tecla F12) y leerlo. Por eso, si pusieras una clave de API en el frontend, la estarías **publicando**: otras personas podrían usarla a tu nombre y agotar tu cupo.

REGLA QUE NO SE ROMPE

Las claves de API **solo** viven en el `.env` del **backend**. En Vite, además, cualquier variable que empiece por `VITE_` es **pública**: nunca pongas secretos ahí.

Por eso el navegador nunca habla directo con Cohere, Supabase ni Gemini: siempre pasa por **nuestro backend**, que es el único que conoce las claves.

3. Repaso corto: RAG y base de datos vectorial

Un **embedding** es una lista de números que captura el *significado* de un texto (Clase 2). Textos parecidos tienen vectores parecidos. Una **base de datos vectorial** guarda esos vectores y sabe encontrar "los más parecidos" a uno dado, usando la **similitud coseno**.

RAG usa esto en tres pasos:

1. **Vectorizar** los documentos una vez (ingesta).
2. **Buscar** (retrieval) los fragmentos más parecidos a la pregunta.
3. **Generar** (generation) la respuesta con el LLM usando SOLO esos fragmentos.

4. Crear el frontend con React + Vite

Vite es la forma moderna y rápida de arrancar un proyecto React. Necesitas tener instalado **Node.js** (versión 18 o superior). Desde una terminal, en la carpeta donde quieras el proyecto:

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev          # arranca en http://localhost:5173
```

La estructura importante que crea Vite:

```
frontend/
├── index.html
├── package.json
├── src/
│   ├── main.jsx      # monta la app en la página
│   ├── App.jsx       # componente principal (aquí va el chat)
│   └── App.css       # estilos
```

CONSEJO

Deja este servidor corriendo en una terminal. Abre **otra terminal** aparte para el backend. Necesitas los dos encendidos al mismo tiempo.

5. Crear el backend con Express

El backend es un servidor pequeño de Node.js con **dos endpoints**: `/ingest` (cargar documentos) y `/chat` (responder preguntas). En una carpeta `backend/` aparte:

```
cd backend
npm init -y
npm install express cors dotenv cohere-ai @google/generative-ai @supabase/supabase-js
```

Para poder usar `import` (módulos ES), añade esta línea en el `backend/package.json`:

```
{
  "type": "module",
  "scripts": { "start": "node server.js" }
}
```

5.1 El archivo `server.js`

Define el servidor, activa **CORS** (para que React pueda llamarlo) y crea los dos endpoints. Archivo completo en `codigo/backend/server.js` :

```
import express from "express";
import cors from "cors";
import "dotenv/config"; // carga el .env
import { ingestarTexto, responderPregunta } from "./rag.js";

const app = express();
app.use(cors()); // permite las llamadas de React
app.use(express.json()); // permite leer JSON del body

// Endpoint para CARGAR documentos a la base vectorial
app.post("/ingest", async (req, res) => {
  try {
    const { texto } = req.body;
    const n = await ingestarTexto(texto);
    res.json({ ok: true, fragmentos: n });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

// Endpoint para PREGUNTAR al agente RAG
app.post("/chat", async (req, res) => {
  try {
    const { pregunta } = req.body;
    const respuesta = await responderPregunta(pregunta);
    res.json({ respuesta });
  } catch (e) {
    res.status(500).json({ error: e.message });
  }
});

app.listen(3000, () => console.log("Backend escuchando en http://localhost:3000"));
```

¿QUÉ ES CORS?

Por seguridad, el navegador bloquea que una página (localhost:5173) llame a otro origen (localhost:3000). La librería `cors` le da permiso explícito a nuestro frontend.

6. Configurar Supabase + pgvector

Supabase es una base de datos PostgreSQL en la nube con plan gratuito. Pasos:

1. Entra a **supabase.com**, crea una cuenta y un **proyecto nuevo** (guarda la contraseña de la base).
2. En el menú lateral abre el **SQL Editor**.
3. Pega y ejecuta el contenido de `codigo/backend/supabase.sql` (lo vemos abajo).
4. Ve a **Project Settings** → **API** y copia la **Project URL** y la clave **service_role**.

El SQL habilita pgvector, crea la tabla y la función de búsqueda:

```
-- 1. Habilitar la extensión de vectores
create extension if not exists vector;

-- 2. Tabla de documentos. 1024 = dimensión de embed-multilingual-v3.0
create table documentos (
  id          bigserial primary key,
  contenido text,
  embedding vector(1024)
);

-- 3. Función que busca los documentos más parecidos (similitud coseno)
create or replace function match_documents(
  query_embedding vector(1024),
  match_count int default 3
)
returns table (id bigint, contenido text, similitud float)
language sql stable
as $$
select
  id,
  contenido,
  1 - (embedding <=> query_embedding) as similitud
from documentos
order by embedding <=> query_embedding -- <=> es distancia coseno
limit match_count;
$$;
```

CLAVE SERVICE_ROLE

Es una clave **muy poderosa** (puede leer y escribir todo). Úsala **solo** en el backend, nunca en React. Va en el `.env` del backend.

7. Ingesta de datos (cargar los documentos)

La ingesta convierte nuestros documentos en vectores y los guarda. Se hace **una sola vez** (o cada vez que agregas material nuevo). Tres pasos: **partir en chunks** → **generar embeddings** → **guardar en Supabase**.

En `codigo/backend/rag.js` (parte de ingesta):

```
import { CohereClient } from "cohere-ai";
import { createClient } from "@supabase/supabase-js";

const cohere = new CohereClient({ token: process.env.COHERE_API_KEY });
const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_KEY);

// Parte un texto largo en fragmentos de ~500 caracteres
function partirEnChunks(texto, tam = 500) {
  const chunks = [];
  for (let i = 0; i < texto.length; i += tam) {
    chunks.push(texto.slice(i, i + tam));
  }
  return chunks;
}

export async function ingestarTexto(texto) {
  const chunks = partirEnChunks(texto);

  // input_type "search_document": textos que se GUARDAN en la base
  const r = await cohere.embed({
    texts: chunks,
    model: "embed-multilingual-v3.0",
    inputType: "search_document",
  });

  // Cada fila = un fragmento + su vector
  const filas = chunks.map((c, i) => ({
    contenido: c,
    embedding: r.embeddings[i],
  }));

  await supabase.from("documentos").insert(filas);
  return chunks.length;
}
```

¿POR QUÉ CHUNKS?

Fragmentos pequeños hacen que la búsqueda sea más precisa (traes justo el pedazo relevante) y que el prompt final sea más corto, lo que sale más barato y rápido.

8. Flujo de consulta: el agente RAG

Cuando el usuario pregunta algo, el backend hace cuatro pasos:

1. Convertir la **pregunta en un embedding** (con `input_type: "search_query"`).
2. Llamar a `match_documents` en Supabase para traer los **chunks más parecidos**.
3. Armar un **prompt** que incluya ese contexto y una instrucción clara.
4. Pedirle a **Gemini** la respuesta y devolverla al frontend.

En `codigo/backend/rag.js` (parte de consulta):

```
import { GoogleGenerativeAI } from "@google/generative-ai";
const genai = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

export async function responderPregunta(pregunta) {
  // 1. Embedding de la PREGUNTA (search_query)
  const e = await cohere.embed({
    texts: [pregunta],
    model: "embed-multilingual-v3.0",
    inputType: "search_query",
  });

  // 2. RETRIEVAL: buscar los 3 chunks más parecidos en Supabase
  const { data, error } = await supabase.rpc("match_documents", {
    query_embedding: e.embeddings[0],
    match_count: 3,
  });
  if (error) throw error;

  const contexto = data.map((d) => `- ${d.contenido}`).join("\n");

  // 3. Armar el prompt con el contexto recuperado
  const prompt = `Responde usando ÚNICAMENTE esta información.
Si la respuesta no está ahí, di que no tienes ese dato.

Información:
${contexto}

Pregunta: ${pregunta}`;

  // 4. GENERATION: Gemini genera la respuesta
  const model = genai.getGenerativeModel({ model: "gemini-1.5-flash" });
  const out = await model.generateContent(prompt);
  return out.response.text();
}
```

EL AGENTE NO INVENTA

Como le pedimos responder SOLO con el contexto, si la información no está en tus documentos, el agente lo dice en vez de inventar. Así se reducen las alucinaciones.

El archivo `.env` del backend

Crea `backend/.env` con tus claves reales (guíate por `codigo/backend/.env.example`):

```
COHERE_API_KEY=tu_clave_de_cohere  
GEMINI_API_KEY=tu_clave_de_gemini  
SUPABASE_URL=https://xxxxx.supabase.co  
SUPABASE_KEY=tu_clave_service_role_de_supabase
```

9. Conectar React con el backend (UI de chat)

En `frontend/src/App.jsx` creamos un chat sencillo con `useState` y `fetch`. Archivo completo en `codigo/frontend/App.jsx`:

```

import { useState } from "react";
import "../App.css";

const API = "http://localhost:3000"; // dirección del backend

export default function App() {
  const [mensajes, setMensajes] = useState([]);
  const [texto, setTexto] = useState("");
  const [cargando, setCargando] = useState(false);

  async function enviar() {
    if (!texto.trim()) return;
    const pregunta = texto;
    setMensajes((m) => [...m, { rol: "yo", texto: pregunta }]);
    setTexto("");
    setCargando(true);
    try {
      const r = await fetch(`${API}/chat`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ pregunta }),
      });
      const { respuesta } = await r.json();
      setMensajes((m) => [...m, { rol: "bot", texto: respuesta }]);
    } catch (e) {
      setMensajes((m) => [...m, { rol: "bot", texto: "Error: " + e.message }]);
    } finally {
      setCargando(false);
    }
  }

  return (
    <div className="chat">
      <h1>Asistente RAG</h1>
      <div className="mensajes">
        {mensajes.map((m, i) => (
          <div key={i} className={`msg ${m.rol}`}>{m.texto}</div>
        ))}
        {cargando && <div className="msg bot">escribiendo...</div>}
      </div>
      <div className="fila">
        <input
          value={texto}
          onChange={(e) => setTexto(e.target.value)}
          onKeyDown={(e) => e.key === "Enter" && enviar()}
          placeholder="Escribe tu pregunta..."
        />
        <button onClick={enviar}>Enviar</button>
      </div>
    </div>
  );
}

```

```
);  
}
```

ORDEN PARA PROBAR

- 1) Ejecuta el SQL en Supabase. 2) Arranca el backend (`npm start`). 3) Carga documentos con `/ingest` (ver README del código). 4) Arranca el frontend (`npm run dev`) y pregunta.

10. Seguridad, límites y despliegue

10.1 Seguridad de las claves — ¡crítico!

1. Todas las claves van en `backend/.env` , **NUNCA** en React.
2. Crea un `.gitignore` para no subir secretos ni dependencias:

```
# .gitignore  
node_modules/  
.env
```

SI UNA CLAVE SE FILTRA

Revócala de inmediato y genera una nueva desde el panel del proveedor (Cohere, Google AI Studio o Supabase).

10.2 Límites y costos

- Las APIs gratuitas tienen **límites de peticiones por minuto** y cupos mensuales.
- Los LLMs cobran o limitan **por token**: trae solo 3–4 chunks para que el prompt sea corto.
- Guarda los embeddings una sola vez (no re-vectorices en cada pregunta).

10.3 Cómo desplegar

Parte	Servicio gratuito	Notas
Frontend (React)	Vercel o Netlify	Conecta tu repo de GitHub; se publica solo
Backend (Express)	Render o Railway	Configura las claves como variables de entorno
Base vectorial	Supabase (ya en la nube)	No necesita despliegue aparte

UN DETALLE AL DESPLEGAR

Cambia la dirección del backend en el frontend: en vez de <http://localhost:3000> , usa la URL pública que te dé Render o Railway.

Actividades de refuerzo

Realiza estas actividades para afianzar lo aprendido. Puedes escribir tus respuestas en los espacios en blanco.

1 Dibuja la arquitectura Fácil

Dibuja el diagrama de la app (React → backend → Cohere / Supabase / Gemini) y explica con tus palabras qué hace cada pieza y por qué las claves no pueden ir en React.

2 Levanta el frontend Fácil

Crea el proyecto con `npm create vite@latest`, arráncalo con `npm run dev` y cambia el título del chat en `App.jsx`. Anota qué dirección (puerto) abrió el navegador.

3 Crea tu Supabase Media

Crea un proyecto en Supabase, ejecuta `supabase.sql` y confirma que la tabla `documentos` y la función `match_documents` aparecen. Anota la dimensión del vector y por qué es 1024.

4 Prueba los endpoints Media

Con el backend corriendo, usa el navegador o una herramienta como Thunder Client/Postman para llamar a `/ingest` con un texto de prueba y luego a `/chat` con una pregunta. Anota la respuesta que recibiste.

5 Afina el retrieval Reto

Cambia `match_count` de 3 a 1 y luego a 5, y cambia el tamaño de los chunks (por ejemplo 300 y 800 caracteres). Anota cómo cambian las respuestas y explica el equilibrio entre precisión y costo.

6 Proyecto final: tu propio agente RAG Reto

Carga **tus propios documentos** (apuntes, el reglamento del SENA, la información de tu emprendimiento...) usando `/ingest`, y haz que el chat responda con ellos. Incluye una pregunta cuya respuesta NO esté en tus documentos para verificar que el agente responde "no tengo ese dato". Recuerda usar `.env` y `.gitignore`. Describe aquí tu idea y qué documentos cargaste:

Lista de verificación

☐ Entiendo la arquitectura React → backend → embeddings + base vectorial + LLM.

- ☐ Sé explicar por qué las claves van en el backend y nunca en React.
- ☐ Creé el frontend con React + Vite y lo puedo arrancar.
- ☐ Creé el backend Express con los endpoints `/ingest` y `/chat` .
- ☐ Configuré Supabase con pgvector, la tabla `documentos` y `match_documents` .
- ☐ Implementé la ingesta (chunks + embeddings) y la consulta (retrieval + Gemini).
- ☐ Conecté el chat de React con el backend usando `fetch` .
- ☐ Guardo mis claves en `.env` y tengo un `.gitignore` .
- ☐ Cargué mis propios documentos y el chat responde con ellos.

PARA SEGUIR APRENDIENDO

Otras bases vectoriales: Pinecone, Upstash Vector, Chroma, FAISS. · **Frameworks RAG:** LangChain, LlamaIndex. · **Despliegue:** Vercel, Netlify, Render, Railway. · **Documentación oficial:** React/Vite, Supabase, Google AI Studio y Cohere.